

ui.circular_progress 全面详解

`ui.circular_progress` 是 NiceGUI 框架中基于 Quasar 组件封装的**环形进度条组件**，专注于直观展示任务进度、加载状态或数值占比（范围 0.0-1.0）。其核心优势是轻量化、视觉聚焦、配置灵活，支持样式定制、值绑定、进度文本自定义等功能，适配需要突出进度展示的场景（如数据加载、任务完成度、百分比指标等）。以下从核心特性、使用场景、详细配置、高级技巧等维度展开全面解析。

一、核心概述

1. 本质与定位

- 基于 Quasar 的 `QCircularProgress` 组件封装，以环形可视化形式呈现进度，相比线性进度条更节省横向空间，视觉上更突出。
- 支持进度值动态更新、样式自定义（颜色、尺寸、厚度）、进度文本定制（百分比 / 自定义文案），适配从简单加载提示到复杂指标展示的各类场景。
- 继承 NiceGUI 组件通用能力，支持值绑定、事件监听、可见性控制，轻松融入 Web 交互逻辑。

2. 核心参数（初始化时必填 / 常用）

参数名	类型	说明
<code>value</code>	float	初始进度值（必填，范围 0.0-1.0，0 表示无进度，1 表示完成）。
<code>size</code>	str / int	环形进度条整体尺寸（直径），默认 <code>"100px"</code> ，支持像素值（如 <code>150</code> ）或 CSS 长度单位（如 <code>"3rem"</code> ）。
<code>thickness</code>	int	环形进度条的线条厚度（像素），默认 <code>5</code> ，值越大线条越粗。
<code>show_value</code>	bool	是否显示中心文本（默认 <code>True</code> ），文本默认显示百分比（如 <code>50%</code> ）。
<code>color</code>	str / None	进度条颜色（默认 <code>"primary"</code> ），支持 Quasar 主题色（如 <code>"secondary"</code> 、 <code>"accent"</code> ）、Tailwind 颜色（如 <code>"bg-blue-500"</code> ）或 CSS 颜色（如 <code>"#b687ac"</code> ），设为 <code>None</code> 时使用默认样式。
<code>bg_color</code>	str	进度条背景色（未填充部分），默认 <code>"grey-2"</code> （Quasar 灰色系），支持 CSS 颜色。
<code>label</code>	str / Callable	中心自定义文本（优先级高于默认百分比），可传入字符串或返回字符串的函数（动态更新文本）。
<code>style</code>	Style[Self]	组件容器样式，支持 Tailwind CSS（如 <code>shadow-md</code> 、 <code>rounded-full</code> ）。
<code>classes</code>	Classes[Self]	组件容器的 HTML 类名，用于样式复用。
<code>visible</code>	bool	初始可见性（默认 <code>True</code> ），支持后续动态修改或绑定。

二、基础使用场景与示例

1. 基础静态环形进度条

直接设置 `value` 参数，快速创建静态环形进度条，适用于固定进度展示（如任务完成度概览、指标占比）。

示例：基础与自定义静态进度条

```
from nicegui import ui

# 基础配置：50% 进度、默认尺寸/厚度、显示百分比文本
ui.circular_progress(value=0.5)

# 自定义配置：70% 进度、150px 尺寸、8px 厚度、绿色、隐藏文本
ui.circular_progress(
    value=0.7,
    size=150,
    thickness=8,
    color='green',
    show_value=False,
    bg_color='#f0f0f0' # 自定义背景色
)

ui.run()
```

2. 动态更新进度（模拟加载）

通过 `set_value` 方法修改进度值，结合 `ui.timer` 模拟实时进度更新（如数据加载、文件上传）。

示例：模拟加载进度（0-100%）

```
from nicegui import ui

# 初始化环形进度条：初始值 0、120px 尺寸、6px 厚度、蓝色
progress = ui.circular_progress(
    value=0.0,
    size=120,
    thickness=6,
    color='#28738a'
)

# 模拟进度更新：每 150ms 增加 3%，直至 100%
current_value = 0.0
def update_progress():
    global current_value
    if current_value < 1.0:
        current_value += 0.03
        progress.set_value(min(current_value, 1.0)) # 更新进度值
    else:
        # 进度完成后修改中心文本为 "Done!"
        progress.label = "Done!"
        timer.deactivate() # 停止定时器

timer = ui.timer(0.15, update_progress)
```

```
# 启动定时器
timer = ui.timer(0.15, update_progress)

ui.run()
```

3. 自定义中心文本（动态文案）

通过 `label` 参数自定义中心文本，支持静态字符串或动态函数（根据进度值自动更新文案）。

示例：动态文本与静态文本

```
from nicegui import ui

# 1. 动态文本：显示进度值 + 自定义后缀（如 "30/100"）
def dynamic_label():
    return f"{int(progress.value * 100)}/100"

progress = ui.circular_progress(
    value=0.3,
    size=100,
    label=dynamic_label, # 传入函数，动态生成文本
    color='purple'
)

# 2. 静态文本：直接显示自定义文案（忽略默认百分比）
ui.circular_progress(
    value=0.8,
    size=100,
    label="80分", # 静态文本
    thickness=5,
    color='orange'
)

# 按钮控制进度更新（验证动态文本同步）
ui.button('Increase Progress', on_click=lambda: progress.set_value(min(progress.value + 0.1, 1.0)))

ui.run()
```

4. 与滑块绑定（实时控制进度）

通过 `bind_value_from` 方法将环形进度条与滑块组件绑定，实现进度的手动调节（如音量控制、任务进度调整）。

示例：滑块控制环形进度条

```

from nicegui import ui

# 创建滑块（范围 0-1，步长 0.01，初始值 0.4）
slider = ui.slider(min=0, max=1, step=0.01, value=0.4)

# 环形进度条绑定滑块值（滑块变化时，进度条自动更新）
ui.circular_progress(
    size=120,
    thickness=7,
    color='teal'
).bind_value_from(slider, 'value')

ui.run()

```

5. 进度变化事件监听

通过 `on_value_change` 方法绑定回调，监听进度值变化（如进度完成时触发提示、执行后续操作）。

示例：进度完成时弹出成功提示

```

from nicegui import ui

progress = ui.circular_progress(
    value=0.0,
    size=100,
    color='green'
)

# 监听进度值变化
def on_progress_change(e):
    current_value = e.value
    if current_value >= 1.0:
        ui.notify('Task completed! 🎉', type='success', position='top-right')

progress.on_value_change(on_progress_change)

# 模拟进度更新
ui.timer(0.2, lambda: progress.set_value(min(progress.value + 0.05, 1.0)))

ui.run()

```

6. 自定义样式（颜色、阴影、圆角）

结合 `color`、`bg_color`、`style`、`classes` 参数，定制环形进度条外观，适配 Web 界面设计风格。

示例：高颜值定制环形进度条

```

from nicegui import ui

# 1. 渐变颜色 + 阴影 + 自定义背景
ui.circular_progress(
    value=0.65,

```

```

size=140,
thickness=8,
# 进度条渐变颜色（通过 classes 实现）
classes=''
    [role="progressbar"] {
        background: linear-gradient(90deg, #3b82f6, #06b6d4);
    }
'',
bg_color='#e5e7eb', # 背景色
style='shadow-lg rounded-full p-2' # 阴影 + 圆角容器
)

# 2. 深色模式适配：深色背景 + 亮色进度条
with ui.card(style='bg-gray-800 text-white p-6'):
    ui.circular_progress(
        value=0.75,
        size=120,
        thickness=5,
        color='#f1c40f', # 黄色进度条
        bg_color='#4b5563', # 深色背景
        label="75%",
        style='margin: 0 auto' # 居中显示
    )

ui.run()

```

7. 可见性绑定（动态显示 / 隐藏）

通过 `bind_visibility` 方法将环形进度条可见性与其他组件状态绑定（如开关、复选框），适配复杂界面交互。

示例：开关控制环形进度条显示

```

from nicegui import ui

# 创建开关组件
show_progress = ui.switch('Show Circular Progress', value=True)

# 创建环形进度条并绑定可见性
progress = ui.circular_progress(
    value=0.5,
    size=100,
    color='red'
)
progress.bind_visibility(show_progress, 'value')

ui.run()

```

8. 多环形进度条并列（多指标展示）

多个环形进度条并列展示，适用于多指标占比监控（如多维度评分、多任务进度）。

示例：多指标环形进度条组

```
from nicegui import ui

# 模拟 4 个指标数据
indicators = [
    {'name': 'Accuracy', 'value': 0.85, 'color': '#28738a'},
    {'name': 'Precision', 'value': 0.78, 'color': '#b687ac'},
    {'name': 'Recall', 'value': 0.92, 'color': '#a78f8f'},
    {'name': 'F1-Score', 'value': 0.88, 'color': '#f1c40f'}
]

# 网格布局: 2x2 排列
with ui.grid(columns=2, gap=20):
    for indicator in indicators:
        with ui.column(align_items='center'):
            # 环形进度条
            ui.circular_progress(
                value=indicator['value'],
                size=80,
                thickness=4,
                color=indicator['color'],
                label=f"{int(indicator['value']*100)}%"
            )
            # 指标名称
            ui.label(indicator['name'], classes='mt-2 font-medium')

ui.run()
```

三、核心属性与方法详解

1. 常用属性

属性名	类型	说明
value	BindableProperty	可读可写，进度值（0.0-1.0），修改后自动刷新进度条（如 <code>progress.value = 0.7</code> ）。
size	str / int	可读可写，环形进度条直径，支持动态修改（如 <code>progress.size = 150</code> ）。
thickness	int	可读可写，进度条线条厚度（像素），修改后需调用 <code>update()</code> 生效（如 <code>progress.thickness = 8; progress.update()</code> ）。
show_value	bool	可读可写，是否显示中心文本，修改后需调用 <code>update()</code> 生效（如 <code>progress.show_value = False; progress.update()</code> ）。

属性名	类型	说明
<code>color</code>	str / None	可读可写，进度条颜色，支持动态修改（如 <code>progress.color = 'green'</code> ）。
<code>bg_color</code>	str	可读可写，进度条背景色，支持动态修改（如 <code>progress.bg_color = '#f0f0f0'</code> ）。
<code>label</code>	str / Callable / None	可读可写，中心自定义文本，支持静态字符串、动态函数或 <code>None</code> （恢复默认百分比）。
<code>visible</code>	BindableProperty	可读可写，控制组件是否可见（支持绑定，如 <code>bind_visibility</code> ）。
<code>html_id</code>	str	组件在 HTML DOM 中的唯一 ID（v2.16.0+ 支持）。

2. 核心方法

方法名	作用	参数说明
<code>set_value(value)</code>	手动设置进度值（核心方法），值需在 0.0-1.0 范围内。	<code>value</code> ：进度值（float，如 0.5 表示 50%）。
<code>on_value_change(callback)</code>	绑定进度值变化事件回调。	<code>callback</code> ：接收 <code>ValueChangeEventArguments</code> 对象， <code>e.value</code> 为当前进度值。
<code>bind_value(target)</code>	双向绑定进度值到目标对象的属性（如滑块、变量）。	<code>target_object</code> ：目标对象； <code>target_name</code> ：属性名（默认 <code>value</code> ）。
<code>bind_value_from(target)</code>	单向绑定进度值（从目标对象到进度条）。	参数同 <code>bind_value</code> ，仅单向同步。
<code>bind_value_to(target)</code>	单向绑定进度值（从进度条到目标对象）。	参数同 <code>bind_value</code> ，仅单向同步。
<code>bind_visibility(target)</code>	双向绑定可见性到目标对象的属性（如开关）。	<code>target_object</code> ：目标对象； <code>target_name</code> ：属性名（默认 <code>visible</code> ）。
<code>update()</code>	手动触发组件刷新（修改 <code>thickness</code> 、 <code>show_value</code> 等属性后需调用）。	-
<code>set_visibility(visible)</code>	直接设置可见性（布尔值）。	<code>visible</code> ： <code>True</code> （显示）/ <code>False</code> （隐藏）。
<code>delete()</code>	删除组件，释放资源。	-

四、高级技巧与注意事项

1. 样式定制进阶

- **渐变进度条**: 通过 `classes` 参数自定义进度条的 `background` 为渐变颜色（参考“自定义样式”示例），需注意选择器 `[role="progressbar"]` 对应进度条本身。
- **容器美化**: 结合 Tailwind CSS 为组件容器添加阴影（`shadow-md`）、圆角（`rounded-full`）、内边距（`p-2`），提升视觉效果。
- **深色模式适配**: 调整 `color` 和 `bg_color` 为高对比度颜色（如亮色进度条 + 深色背景），确保在深色模式下可读性。

2. 动态文本高级用法

- **条件文本**: 在 `label` 函数中根据进度值返回不同文案（如进度 < 0.5 显示“加载中”， ≥ 1.0 显示“完成”）：

```
def conditional_label():
    if progress.value < 0.5:
        return "Loading..."
    elif progress.value < 1.0:
        return f"{int(progress.value*100)}%"
    else:
        return "Done!"
progress.label = conditional_label
```

- **HTML 文本**: 若需支持 HTML 格式文本（如换行、字体样式），可通过 `style` 或 `classes` 间接实现，或结合 `ui.label` 嵌套（需自定义组件）。

3. 性能优化

- **高频更新控制**: 若需高频更新进度（如每秒 10 次以上），可适当降低更新频率，或使用 `throttle` 限制事件触发次数：

```
progress.on_value_change(callback, throttle=0.1) # 0.1 秒内仅触发一次
```

- **避免不必要刷新**: 修改 `value` 时直接赋值或调用 `set_value`，无需额外调用 `update()`；修改 `thickness`、`show_value` 等属性后才需调用 `update()`。

4. 兼容性与版本说明

- **颜色兼容性**: `color` 和 `bg_color` 支持 Quasar 主题色、Tailwind 颜色、CSS 颜色，建议优先使用 CSS 颜色（如 `#xxx`、`rgb()`）确保跨环境兼容。
- **版本要求**: `html_id` 属性需 NiceGUI v2.16.0+ 支持，低版本需避免使用；`label` 参数的函数式用法支持所有版本。
- **文本显示限制**: 中心文本长度不宜过长，否则会超出环形范围，建议控制在 5 个字符以内（或缩小字体，需通过 `style` 自定义）。

5. 常见问题排查

- **进度条不更新**: 检查 `value` 是否在 0.0-1.0 范围内, 修改属性后是否调用 `update()` (如 `thickness` 变化后)。
- **颜色不生效**: 确认颜色值格式正确 (如 Tailwind 颜色需带 `bg-` 前缀, CSS 颜色需符合语法), 避免拼写错误。
- **文本显示异常**: `show_value=True` 时, 若 `label` 为 `None` 则显示百分比; 若设置了 `label` 则优先显示自定义文本; 文本过长时可缩小 `size` 或调整字体大小 (通过 `style` 自定义)。
- **环形变形**: `size` 建议使用正方形尺寸 (如 `100px`、`150`), 避免使用非对称值 (如 `100px x 80px`) 导致环形变形。

五、与 `ui.linear_progress` 的核心区别

`ui.circular_progress` 与 `ui.linear_progress` 均为进度展示组件, 但适用场景与视觉效果存在差异, 选择时可参考下表:

特性	<code>ui.circular_progress</code>	<code>ui.linear_progress</code>
视觉形态	环形 (节省横向空间, 视觉突出)	线性 (横向延伸, 适合批量排列)
核心优势	聚焦单个指标、视觉冲击力强、节省空间	适合多任务并列、横向空间充足、进度对比直观
文本定制	支持中心自定义文本 (静态 / 动态)	仅支持中心百分比文本 (默认)
样式定制	支持环形厚度、背景色、渐变颜色	支持高度、圆角、渐变颜色
适用场景	单个任务进度、指标占比、加载状态	多任务进度并列、文件上传、流程步骤进度

六、总结

`ui.circular_progress` 是 NiceGUI 中专注于**环形进度可视化**的轻量化组件, 核心优势在于:

1. 视觉聚焦: 环形形态突出单个进度指标, 相比线性进度条更具视觉冲击力。
2. 配置灵活: 支持尺寸、厚度、颜色、背景色、自定义文本等全方位样式定制。
3. 动态交互: 支持进度值更新、事件监听、组件绑定, 适配实时监控场景。
4. 空间高效: 节省横向空间, 适合在卡片、仪表盘等紧凑布局中使用。
5. 易用性强: 通过少量参数即可实现复杂功能, 无需编写前端代码。

适用于数据加载、文件上传、任务完成度展示、多指标占比监控、仪表盘指标等场景, 是 Web 应用中需要突出进度或指标的优选组件。